Autómata Lab

Documento técnico y funcional

Microproyecto 3 · Teoría de la Computación · 2026/02

1. Propósito y alcance

El software permite estudiar cómo distintas representaciones describen el mismo lenguaje regular. A partir de una expresión regular o de un autómata manual, obtiene un AFD mínimo y permite revisar las operaciones realizadas, su tabla, su diagrama y sus simulaciones.

El público son estudiantes y docentes de Teoría de la Computación. Las dos modalidades comparten el mismo motor Python. Se admiten AFD completos, AFD incompletos, AFN y AFN con transiciones ε. La aplicación se ejecuta localmente o como servicio web en Render; no requiere cuentas de usuario ni una base de datos.

Se excluyen expresiones regulares avanzadas de programación (capturas, referencias, rangos y lookahead), conversión a autómatas de pila, máquinas de Turing, almacenamiento de sesiones y edición gráfica por arrastre. La representación gráfica acompaña las transformaciones; la tabla es la referencia exacta para revisar cada transición.

El documento de referencia tiene una errata en RF-06: se implementa AFN → AFD, conforme a los objetivos. RF-01 queda incompleto después de «reservando»; esta implementación reserva ε, ∅ y los operadores. No existe RF-08 en la lista original. Python fue el lenguaje solicitado para esta entrega.

2. Casos de uso

Caso	Entrada y acción	Resultado observable
CU-01 Construir	Registrar alfabeto y expresión; pulsar Construir y minimizar	Construcción Thompson, accesibilidad, AFD completo y AFD mínimo
CU-02 Registrar	Registrar estados, inicial, finales y tabla de destinos	Clasificación justificada y procesamiento del autómata
CU-03 Explorar	Seleccionar una de las cuatro etapas	Tabla, diagrama, correspondencias y operaciones concretas
CU-04 Simular	Elegir etapa y escribir una cadena	Conjunto de estados por paso y aceptación o rechazo
CU-05 Comparar	Abrir comparación con el AFD mínimo	Número de estados y tabla final, sin perder la etapa seleccionada
CU-06 Exportar	Descargar resultado o SVG	JSON de todas las etapas o diagrama de la etapa actual

Si la entrada es inválida se informa qué debe corregirse y no se entrega un resultado parcial. Si el autómata ya es determinista se indica expresamente y se omite la construcción por subconjuntos. Al editar la entrada se descartan los resultados anteriores para evitar asociarlos a datos nuevos.

3. Sintaxis, entradas y restricciones

El alfabeto es una lista no vacía de hasta 12 caracteres Unicode visibles. Cada símbolo ocupa un carácter; no se permiten duplicados, espacios, coma, barra inversa, paréntesis, |, *, ε ni ∅. En la interfaz se escribe, por ejemplo, a, b; en la API, ["a", "b"].

Gramática de expresiones:

```
expresion := concatenacion ("|" concatenacion)*
concatenacion := repeticion+
repeticion := atomo ("*")*
atomo := simbolo | "ε" | "∅" | "(" expresion ")"
```

La prioridad descendente es estrella, concatenación y unión. ab|c significa (ab)|c. ε representa únicamente la cadena vacía y ∅ el lenguaje sin cadenas. Las expresiones no admiten espacios ni operandos implícitamente vacíos: use a|ε, no a|. Los caracteres + y ? son literales únicamente si se incluyen en el alfabeto; no son operadores. La longitud máxima de expresión es 120 caracteres.

Los autómatas manuales admiten de 1 a 80 estados y hasta 4000 aristas. Los nombres comienzan con letra ASCII o _, seguidos de letras ASCII, números, _ o -, hasta 32 caracteres. Hay exactamente un estado inicial y cualquier cantidad de finales, incluso cero. Los destinos son listas sin duplicados de estados existentes. Una celda sin destinos representa ausencia de transición, no una transición ε.

La construcción interna admite 256 estados; la determinización se detiene con un error explícito antes de superar 128. Las cadenas tienen hasta 1000 caracteres. En el simulador, el campo vacío representa ε; escribir el símbolo ε literal se rechaza porque no pertenece al alfabeto. Las solicitudes HTTP se limitan a 256 KiB. Estos límites son educativos y protegen frente al crecimiento exponencial de la determinización.

4. Arquitectura y estructuras de datos

La aplicación tiene tres capas: motor matemático independiente en Python, adaptador HTTP Flask e interfaz HTML/CSS/JavaScript. El motor puede probarse sin navegador ni servidor. JavaScript gestiona formularios y SVG, pero las transformaciones y simulaciones se calculan en Python.

Automaton es una dataclass con states y alphabet como tuplas ordenadas, initial como cadena, finals como frozenset y transitions como diccionario de origen a símbolo a frozenset de destinos. Incluso en un AFD cada destino es un conjunto de un elemento. La serialización JSON convierte conjuntos a listas ordenadas; ε se representa con la clave "ε".

La cola de accesibilidad y la de determinización usan deque. Un subconjunto se almacena como frozenset, lo que permite usarlo como clave del diccionario que asigna nombres D0, D1, etc. Los bloques de Moore son tuplas de estados y sus firmas son tuplas de índices de bloques.

Cada función devuelve un autómata nuevo o conserva el original cuando no necesita cambiarlo; no modifica los datos recibidos. Los registros de operaciones son listas de texto producidas durante los cálculos. No hay estado mutable compartido entre solicitudes. La interfaz conserva el resultado en memoria de la pestaña; al recargar se pierde, aunque puede descargarse como JSON.

5. Construcción de Thompson

El analizador de descenso recursivo genera un árbol de operaciones según la gramática. Cada nodo se transforma en un fragmento con una entrada y una salida. Para un literal se crean dos estados y una arista con ese símbolo. Para ε se usa una arista vacía; para ∅ se crean dos estados sin conectarlos.

La concatenación conecta por ε la salida del primer fragmento con la entrada del segundo. La unión crea una entrada que bifurca por ε y una salida que reúne ambos fragmentos. La estrella crea caminos para cero repeticiones, para entrar al fragmento, para repetirlo y para salir. Cada conexión queda descrita en la traza, con sus estados concretos.

La corrección se justifica por inducción sobre el árbol: los fragmentos base reconocen su símbolo, ε o ningún lenguaje; las conexiones realizan respectivamente producto, unión y cierre de Kleene. Por ello el fragmento final reconoce exactamente la expresión. El análisis y la construcción son lineales en la longitud de la expresión, salvo el coste de construir las explicaciones.

6. Clasificación y accesibilidad

Si existe alguna transición ε, la entrada es AFN-ε. Sin ε, si alguna pareja estado/símbolo tiene más de un destino, es AFN. Si cada pareja tiene exactamente un destino, es AFD completo. Si ninguna tiene más de uno pero algunas no tienen destino, se informa AFD incompleto.

Una búsqueda desde el inicial recorre todas las aristas, incluidas ε. Los estados no visitados se eliminan junto con sus transiciones y su condición de finales. El lenguaje no cambia porque ninguna ejecución iniciada en el estado inicial puede llegar a ellos. El coste es O(n + e), con n estados y e aristas. La clasificación de entrada se conserva aunque al eliminar inaccesibles desaparezca la única fuente de no determinismo.

7. Determinización y completado

El cierre-ε(S) contiene los estados alcanzables desde S mediante cero o más transiciones ε. Se calcula con un conjunto de visitados, por lo que los ciclos ε terminan. El estado inicial del AFD es cierre-ε({inicial}). Para cada subconjunto T y símbolo a se calcula:

```
mover(T, a) = unión de los destinos por a de cada estado de T
δAFD(T, a) = cierre-ε(mover(T, a))
```

Se crean solo subconjuntos alcanzables, en orden BFS; D0 es el inicial. Un subconjunto es final si intersecta los finales originales. El conjunto vacío es un estado no final y sus transiciones vuelven a él. Se conserva el diccionario D → subconjunto original.

El invariante es que, después de leer cualquier prefijo, el estado del AFD contiene exactamente los posibles estados del AFN tras consumirlo y cerrar por ε. Por tanto ambos aceptan las mismas cadenas. Si el AFN tiene n estados pueden aparecer hasta 2^n subconjuntos. Una cota conservadora del coste de esta implementación es O(k · 2^n · (n + e)), con k símbolos, al calcular movimientos y cierres sin precalcular todos los cierres.

Si el autómata accesible ya es determinista, se omite la construcción por subconjuntos. Si está incompleto se crea un estado sumidero no final, se redirigen a él las transiciones ausentes y se añaden sus bucles. Su nombre evita colisiones y su correspondencia original es vacía porque es un estado nuevo. Completar no cambia el lenguaje: las ejecuciones antes indefinidas siguen rechazándose.

8. Minimización de Moore

Se parte de P0 formada por estados finales y no finales, omitiendo bloques vacíos. Para cada estado se construye la firma de los bloques destino, siguiendo el orden del alfabeto. Dentro de cada bloque se agrupan estados con la misma firma. Se repite hasta que la partición no cambia.

Cada separación revela una diferencia observable: inicialmente la aceptación de ε y después la aceptación de alguna continuación. En el punto fijo, estados de un mismo bloque tienen igual aceptación y transiciones a los mismos bloques, de modo que son equivalentes para cualquier continuación. Como el AFD era accesible y

completo, su cociente por estas clases es un AFD mínimo.

El algoritmo registra cada firma y cada partición. El cociente se enumera por BFS desde el bloque inicial: M0 siempre es inicial. Se mantienen M → estados del AFD y M → estados de entrada. Una cota del algoritmo es O(k · n²), con hasta n refinamientos que inspeccionan n estados y k símbolos. Las trazas añaden memoria proporcional al número de operaciones registradas.

9. Ejemplos desarrollados

9.1 Expresión (a|b)*abb

El lenguaje contiene las cadenas binarias sobre {a,b} cuyo sufijo es abb. La construcción genera 14 estados; todos son accesibles. La determinización alcanza 5 subconjuntos y Moore obtiene 4 clases.

Las particiones comienzan separando {D0,D1,D2,D3} de {D4}. D3 se separa porque con b alcanza el bloque final. Después se separa D1 porque con b alcanza D3. D0 y D2 permanecen equivalentes. Las clases son M0={D0,D2}, M1={D1}, M2={D3}, M3={D4}.

Estado	a	b	Interpretación
→ M0	M1	M0	No hay prefijo útil del sufijo abb
M1	M1	M2	El sufijo actual es a
M2	M1	M3	El sufijo actual es ab
* M3	M1	M0	El sufijo actual es abb

aabb recorre M0 → M1 → M1 → M2 → M3 y se acepta. abba termina en M1 y se rechaza. ε se rechaza. Los cuatro estados son distinguibles: M3 se distingue por ε; M2 de M0/M1 por b; M1 de M0 por bb.

9.2 AFN manual: cadenas que terminan en ab

Estados {q0,q1,q2}, inicial q0, final q2. q0 tiene a → {q0,q1} y b → {q0}; q1 tiene b → {q2}; el resto no tiene destinos. Es AFN porque (q0,a) tiene dos destinos.

Los subconjuntos alcanzables son D0={q0}, D1={q0,q1} y D2={q0,q2}. Solo D2 es final. La tabla determinista es: D0: a→D1,b→D0; D1: a→D1,b→D2; D2: a→D1,b→D0. Ya es mínimo y se renombra M0,M1,M2. ab y aab se aceptan; aba se rechaza.

9.3 AFD con equivalentes e inaccesible

Estados {A,B,C,X}, inicial A, finales {B,C}. A, B y C tienen a→B,b→C. X tiene bucles con ambos símbolos. Desde A nunca se visita X, por lo que se elimina.

P0={{A},{B,C}}. B y C tienen la misma firma y permanecen juntos. El cociente tiene M0={A}, no final, y M1={B,C}, final. Ambos símbolos llevan de M0 a M1 y de M1 a sí mismo. Se reconoce toda cadena no vacía sobre {a,b}. ε se rechaza.

10. Interfaz HTTP y representación gráfica

Ruta	Contrato
GET /	Página HTML de la aplicación
GET /health	HTTP 200, objeto con status igual a ok
POST /api/process	Objeto con mode regex, alphabet y expression; o mode manual y automaton
POST /api/simulate	Objeto con automaton y word; devuelve word, accepted y path

automaton contiene states, alphabet, initial, finals y transitions. Cada fila de transitions es un objeto símbolo → lista de destinos. Consulte los tres archivos JSON de examples/ para solicitudes completas. process devuelve classification, summary y cuatro stages: input, reachable, dfa y minimal. Cada etapa contiene automaton, steps y mapping; la mínima añade original_mapping y partitions. La simulación devuelve una entrada de path para la posición cero y otra por símbolo consumido.

Una entrada inválida devuelve HTTP 400 con error. Una solicitud demasiado grande devuelve 413. No se utiliza eval, no se interpreta código enviado por el usuario y las etiquetas de diagramas y tablas se escapan antes de insertarse en HTML. Las claves de estados en el cliente usan objetos sin prototipo donde se construyen diccionarios.

Los diagramas son SVG producidos por JavaScript a partir de los mismos datos de la tabla. Los estados se disponen circularmente, las aristas opuestas se curvan, los bucles tienen trazado propio y los símbolos que comparten origen y destino se agrupan. Una flecha marca el inicial y un doble círculo los finales. Los nombres largos se abrevian en el círculo y se conservan completos en el título accesible y la tabla. Zoom y desplazamiento permiten revisar grafos mayores; los grafos densos pueden requerir consultar la tabla para evitar ambigüedad visual.

11. Dependencias y operación

Flask sirve HTML, recursos y API; Gunicorn ejecuta el servidor en Linux/Render. HTML, CSS y SVG usan capacidades nativas del navegador, sin bibliotecas gráficas externas, CDN ni Graphviz. Pytest se utiliza solo para verificar. ReportLab genera los PDF; pypdf permite inspeccionarlos y pypdfium2 renderizarlos durante revisión. Ninguna dependencia implementa las transformaciones de autómatas.

El despliegue usa render.yaml, Python 3.13.9, el comando de instalación de requirements.txt y Gunicorn enlazado a 0.0.0.0:$PORT. /health comprueba que el proceso responde. Los logs del servidor quedan disponibles en Render. No hay migraciones, secretos, almacenamiento permanente ni servicios adicionales. Las instrucciones completas están en README.md.

12. Estrategia de validación

La matriz se genera a partir del XML real de pytest, sin inventar resultados. Un oráculo independiente explora configuraciones (estado, posición), incluidos ciclos ε. Para los ejemplos y 30 AFN generados con semilla 202602, se enumeran todas las cadenas hasta longitud 6 y se comparan todas las etapas. El ejemplo abb se compara además con el predicado de sufijo hasta longitud 8.

La equivalencia entre AFD completo y mínimo se comprueba exactamente mediante BFS del producto: una pareja con aceptación distinta sería un contraejemplo. La minimalidad se verifica buscando una cadena que distinga cada par de estados diferentes. Estos controles aportan más evidencia que comprobar únicamente el número de estados

o unas cadenas concretas.

Se verifican errores de entrada, límites, trazas, reproducibilidad, API y solicitudes concurrentes. La prueba de Gunicorn solo corre en Linux; GitHub Actions instala desde cero y ejecuta toda la suite. La revisión visual web no pudo completarse en esta sesión: el control de escritorio detuvo el acceso al no poder verificar la URL activa. Esa limitación no se registra como prueba aprobada. Los documentos PDF se renderizan para su revisión por separado.

12.1 Trazabilidad con el enunciado

Requisito	Implementación	Evidencia
RF-01	Validación de alfabeto y símbolos reservados	test_alphabet_validation
RF-02	Analizador de expresiones y pertenencia al alfabeto	test_regex_validation, test_regex_precedence
RF-03	Construcción propia de Thompson	test_suffix_abb, test_empty_word_language
RF-04	Registro de estados y tabla de transiciones	test_manual_nfa, test_manual_validation
RF-05	Clasificación justificada, incluido AFD incompleto	test_partial_sink, test_epsilon_cycle
RF-06	Conversión AFN a AFD	test_manual_nfa, test_generated_automata
RF-07	Búsqueda y eliminación de inaccesibles	test_equivalent_unreachable
RF-09	Refinamiento de Moore y cociente	test_equivalent_unreachable, test_already_minimal_and_roundtrip
RF-10	Trazas de construcción, movimientos y particiones	test_traces_mapping_stability
RF-11	Tabla y diagrama SVG del AFD mínimo	check_frontend.cjs; revisión visual en Chrome (V01)
RF-12	Simulación con recorrido y aceptación	test_simulation_validation, test_process_and_simulate
RF-13	Cuatro etapas consultables y comparación final	test_traces_mapping_stability, check_frontend.cjs; revisión visual en Chrome (V01)

La ejecución Linux registrada en docs/evidencias/ci-linux.json terminó con 22 pruebas aprobadas, incluido Gunicorn, y cuatro comprobaciones de formato del cliente. Estas últimas se ejecutan con un DOM mínimo de prueba; además se revisó la interfaz en Chrome de escritorio (V01, capturas en docs/evidencias/capturas/). El diseño móvil se revisó con 657 px de ancho visible, el mínimo que permite la ventana de Chrome (V02). Los datos de ejecución, enlace y commit verificado se conservan para auditoría.

13. Fuentes

Enunciado proporcionado: MICROPROYECTO3-AF.pdf, «Microproyecto #3», tres páginas, lineamientos 2026/02.

Documentación de Flask: https://flask.palletsprojects.com/en/stable/

Documentación de Render: https://render.com/docs/deploy-flask ; https://render.com/docs/blueprint-spec ; https://render.com/docs/python-version

Los procedimientos de Thompson, subconjuntos y Moore están expuestos y justificados en este documento; su implementación y sus pruebas pueden inspeccionarse en el código fuente.